DOCUMENTO TÉCNICO EJECUTIVO · WEB, SERVIDOR, BASE DE DATOS Y ESP32

Recorrido del código y herramientas

Control de Inserción de Trama · 7mo Informática B, Instituto Leonardo Murialdo · 09/11/2026

Integrantes: Manfredo Juan Ignacio (Project Manager), Lede Buschiazzo Lautaro, Ferrando Jaco, Iannone Mia y Montenegro Franco; con Funes Muriel (7mo Multimedios).

Este documento cuenta cómo está armado el sistema por dentro: qué hace cada parte del código y en qué archivos está, cómo viaja una orden desde el celular hasta el telar (y un aviso desde la botonera hasta la pantalla) y con qué herramientas se hizo. Todos los archivos nombrados están en el repositorio de GitHub; el detalle de cada parte, en los README de cada carpeta.

1. Las partes del sistema y cómo se relacionan

El sistema tiene cuatro partes. La **página web** es lo que usan el operario y el cliente en el celular. El **servidor** guarda las reglas: quién puede hacer qué, en qué estado está el telar y por qué fila va el tejido. La **base de datos** guarda todo lo que tiene que durar. El **ESP32**, en el gabinete del telar, es el que toca la máquina. La web y el ESP32 nunca se hablan entre sí: los dos le hablan al servidor por HTTPS con JSON, y el servidor es el único que lee y escribe la base.

Parte	Dónde corre	Qué hace	Archivos principales
Página web	En el navegador; la sirve el mismo servidor	Ingreso, editor de dibujos, biblioteca, control del telar, ficha en PDF y vista del cliente	`backend/public/index.html`, `styles.css` y `vendor/`
Servidor	Render (Node.js 20 a 22)	API en `/api`, sesiones, permisos, estado del telar y cálculo de la posición	`backend/src/server.js`, `routes/`, `controllers/`, `middleware/`, `utils/`, `nivel2/`
Base de datos	Neon (PostgreSQL)	Dibujos, telares, producción, usuarios, errores y claves de los clientes	`backend/src/db/schema.sql` y las 24 migraciones
Hardware (ESP32)	ESP32 en el gabinete	Lee la botonera y el sensor, pulsa los relés y selecciona las bobinas	`esp32/control_trama_esp32/` (el `.ino` y cinco `.h`)

2. La página web (Frontend)

Es una sola página, `backend/public/index.html` (unas 4.600 líneas de HTML y JavaScript), con su hoja de estilos `styles.css` y las librerías de `vendor/`. No usa ningún framework: el JavaScript arma la pantalla directamente, sin recargar, entre Inicio, Editor y Biblioteca, más las capas de ingreso, la vista del cliente y los diálogos.

- **Ingreso.** Dos pestañas, «Personal» y «Clientes». El personal entra con la huella (WebAuthn): el celular firma un desafío y el servidor verifica la firma, así que la huella nunca sale del teléfono; si no anda, entra con el usuario y el código de recuperación. El cliente entra con la clave de su tejido. La página revisa los campos antes de enviarlos y el servidor vuelve a validarlos.
- **Operación principal.** `renderGrid()` dibuja la grilla del editor: cada columna es una bobina y cada fila una pasada, con sus repeticiones (×) y los intercalados (⇄). El dibujo se guarda solo, como JSON. Los botones ▶ ⏸ ⏪ ⏹ mandan `POST /api/telares/:id/...` y cada 4 segundos la

página consulta el estado del telar. Todo pasa por `apiFetch()`, que usa `fetch()` con la cookie de sesión.

- **Reportes / Comprobantes.** La ficha técnica se arma en el navegador con jsPDF: el telar dibujado, la vista previa de la tela a escala, la producción y el hilado. La biblioteca muestra las estadísticas de cada dibujo.
- **Detalles comunes.** Diseño para celular con botones grandes, modo oscuro, tipografías IBM Plex Sans, IBM Plex Mono y Questrial, avisos cortos (toasts) y errores traducidos a mensajes claros por `apiFetch()`.

3. El servidor (Backend)

Está hecho en Node.js con Express. `server.js` arma la cadena por la que pasa cada pedido: cabeceras de seguridad (`helmet`), compresión, CORS, lectura del JSON, registro (`morgan`) y un límite de pedidos por minuto (`express-rate-limit`). Después cada ruta de `routes/` llama a su controlador de `controllers/`, y al final `errorHandler.js` responde siempre con un mensaje entendible. Al arrancar, `migrator.js` aplica las migraciones que falten.

Rutas	Para qué
`/api/auth`	Sesión, registro y entrada con huella (@simplewebauthn/server), recupero con el código, modo invitado e invitaciones para registrar a alguien nuevo.
`/api/patrones`	Los dibujos: listar, crear, cambiar, borrar, metros por pasada, hilado, estadísticas y la clave del cliente.
`/api/telares`	El telar: crearlo, su estado, cargar un dibujo, pausar, reanudar, avanzar, retroceder, terminar, confirmar la posición, su historial, el control desde la página (solo sensado), los avisos de la botonera (`evento-fisico`) y la validación del conteo. Para el Nivel 2, `patron-actual` y `pasadas`.
`/api/historial`	La producción: cada tanda de tejido con sus fechas y sus pasadas.
`/api/errores`	Los avisos de error, también los que manda el ESP32.
`/api/cliente`	Entrar con la clave del tejido, ver el tejido y salir.

- **Quién es quién (Permisos).** `middleware/auth.js` distingue cuatro accesos: el operario, con una cookie de sesión firmada con HMAC-SHA256; el invitado, que no puede mover el telar; el ESP32, con la cabecera `X-Device-Key`; y el cliente, con su propia cookie, que solo abre `/api/cliente`.
- **Secretos.** No hay contraseñas: el personal entra con la huella. Los códigos de recuperación y las claves de los clientes se guardan con un resumen SHA-256 y cifrados con AES-256-GCM. `SESSION_SECRET`, `RECOVERY_SECRET`, `ESP32_DEVICE_KEY` y `DATABASE_URL` van en el archivo `.env` (en Render, como variables de entorno), que `.gitignore` deja fuera de GitHub; en el repositorio solo está `.env.example`.
- **Validación y cálculos.** `utils/validacion.js` rechaza lo que no cumple (de 1 a 300 filas, de 1 a 8 columnas, una bobina por fila, intercalados sin superponerse). `utils/posicion.js` calcula en el servidor la fila y la pasada a partir de las pasadas contadas. Todas las consultas usan parámetros (`$1`, `$2`) de la librería `pg`, y las operaciones del telar van en transacciones.

4. La base de datos

Es **PostgreSQL**, alojada en **Neon**. `schema.sql` crea las tablas una sola vez (`npm run init-db`) y las migraciones 001 a 024 la fueron cambiando a medida que se sumaron funciones; la tabla

`migraciones_aplicadas` recuerda cuáles ya corrieron. `database/01_base_de_datos_completa.sql` junta todo en un solo archivo, generado por un script, para crear la base de una vez.

Tabla	Qué guarda
`patrones`	Los dibujos: la matriz, las pasadas de cada fila, los colores, los intercalados, los metros por pasada, el hilado y si lo hizo un invitado.
`telares`	Cada telar: su estado (tejiendo, pausado o detenido) y por qué, el dibujo cargado, el control desde la página y el último contacto del ESP32.
`historial_produccion`	Cada trabajo: cuándo empezó y terminó, la fila y la pasada donde va, las pasadas contadas y si el conteo ya se validó.
`usuarios`	El personal: nombre, usuario y el código de recuperación (resumido y cifrado).
`credenciales_biometricas`	La parte pública de cada huella registrada (WebAuthn). La huella en sí queda en el celular.
`desafios_webauthn`	Los desafíos de un solo uso para registrar una huella o entrar con ella.
`invitaciones`	Los códigos de invitación, de un solo uso, para registrar a alguien nuevo.
`errores_log`	Los errores y avisos de la web, del servidor y del ESP32.
`accesos_cliente`	La clave de cada tejido para su cliente, el nombre del cliente y los metros que encargó.
`migraciones_aplicadas`	Qué migraciones ya se aplicaron, y cuándo.

5. Integraciones externas o Hardware (ESP32)

El proyecto no cobra en línea, así que no usa pasarela de pagos ni webhooks. Su integración es el hardware: el firmware del ESP32 está escrito en C++ con Arduino (`esp32/control_trama_esp32/`, el `.ino` y cinco `.h`), y `NIVEL2_INSTALADO`, en `config.h`, decide si además de la botonera se usan el sensor de pasada y la selección de bobinas.

- **Red.** Una tarea consulta el estado del telar cada 2,5 s (`GET /api/telares/:id`), manda los avisos de la botonera (`POST evento-fisico`) y, en el Nivel 2, baja el dibujo (`GET patron-actual`) y reporta las pasadas (`POST pasadas`).
- **Relés y sensores.** Tres relés «aprietan» Marcha, Pausa y Retroceder durante 300 ms; el sensado de esos botones entra por interrupciones con antirrebote. En el Nivel 2, `sensor_pasada.h` cuenta las pasadas y `seleccion_dibujo.h` activa los relés LCA110 de la fila.
- **Misma cuenta que el servidor.** `posicion_dibujo.h` hace el mismo cálculo que `utils/posicion.js`; se compararon en 2,4 millones de pasadas al azar, adelante y atrás.
- **Si se corta el Wi-Fi.** No acciona nada: los relés quedan sueltos y la botonera sigue andando a mano. El watchdog reinicia la placa si algo se cuelga y los errores se mandan cuando vuelve la red.

6. El recorrido de punta a punta

Caso A · El operario toca ▶️ en el celular

1. La **página** manda `POST /api/telares/:id/asignar-patron` con el dibujo (o `/reanudar` si es el mismo), con la cookie de sesión.
2. El **servidor** verifica que sea un operario y que el control desde la página esté prendido.

3. En una transacción, la **base** pasa el telar a «tejiendo» en `telares` y abre el trabajo en `historial_produccion`.
4. El **ESP32** lo ve en su consulta siguiente (a lo sumo 2,5 s) y pulsa el relé de Marcha: la máquina arranca. En el Nivel 2 reporta cada pasada y selecciona la bobina de la fila.
5. La **pantalla**, que consulta cada 4 segundos, muestra «Tejiendo…» y «Fila X / N», con la pasada de la fila, y marca la fila en la grilla.

Caso B · Alguien aprieta Pausa en la botonera

1. La pausa le llega a la máquina directo, como siempre; el **ESP32** la detecta por una interrupción del sensado, con antirrebote.
2. El ESP32 manda `POST /api/telares/:id/evento-fisico` con el tipo «pausa» y la cabecera `X-Device-Key`.
3. El **servidor** comprueba la clave del dispositivo; con otra clave, o sin ella, el pedido se rechaza y no se guarda nada.
4. En la **base**, el telar pasa a «pausado» y queda registrado que la pausa vino de la botonera.
5. La **pantalla**, en su consulta siguiente, avisa «El telar se pausó desde la botonera o desde otra pantalla» y deja de avanzar.

7. Herramientas utilizadas

Herramienta	Para qué se usó
Lenguajes	JavaScript en la página web y en el servidor; HTML y CSS en la página; SQL en la base de datos; C++ (Arduino) en el firmware del ESP32; Python en las simulaciones del firmware.
Backend y Frontend	Node.js 20 a 22 con Express 4 (`pg`, `@simplewebauthn/server`, `helmet`, `cors`, `compression`, `express-rate-limit`, `morgan`, `dotenv`). Página sin framework, con jsPDF y SimpleWebAuthn. Firmware: Arduino IDE 2, core ESP32 y ArduinoJson.
Base de datos y Hosting	PostgreSQL en Neon (nube). El servidor y la página se publican juntos en Render. El modelo 3D del gabinete se hizo con Three.js (`modelado_3d/`) y se ve en el navegador.
GitHub y Pruebas (QA)	Repositorio en GitHub: los cambios se hacen en una rama de trabajo y pasan a `main` por un pull request, donde se revisan; lo corregido en cada revisión quedó anotado en `CAMBIOS_REVISION.md`. Pruebas: `npm test`, integración contra PostgreSQL, el firmware probado en la PC con g++ y simulaciones en Python; en `pruebas_qa/`, ESLint, html-validate y Playwright en tamaño de celular.
Inteligencia Artificial	Claude (Anthropic) en Claude Code, con el plan Claude Pro y el modelo **Claude Opus 5.5**. Asistió en revisiones línea por línea del código y de los documentos, en la programación y las pruebas de funciones (por ejemplo, el modo solo sensado y la vista para los clientes) y en la redacción de los documentos y los manuales. Cada cambio quedó en commits propios en GitHub; el equipo los revisó, corrió las pruebas y probó el sistema en el telar antes de pasarlos a la rama principal.